

Guia de Deploy na AWS

Sistema SaaS - frontend React/Vite, backend Node/Express e PostgreSQL

Objetivo: subir o sistema para producao mantendo o login proprio atual, com infraestrutura simples, segura e preparada para crescer.

Arquitetura Recomendada

Area	Servico recomendado
Frontend	AWS Amplify Hosting para hospedar o React/Vite com CDN, HTTPS e deploy via GitHub.
Backend	AWS App Runner para rodar a API Node/Express com logs, HTTPS, deploy automatico e escala gerenciada.
Banco	Amazon RDS for PostgreSQL, compativel com Prisma e com backups/snapshots gerenciados.
Dominio e SSL	Route 53 para DNS e certificados gerenciados pelo ACM/dominio customizado dos servicos.
Segredos	SSM Parameter Store ou Secrets Manager para DATABASE_URL, JWT_SECRET e credenciais sensiveis.
Logs	CloudWatch para logs e alertas do backend.
Arquivos futuros	S3 para comprovantes, PDFs, anexos, imagens e exportacoes.

Fluxo de alto nivel

- Usuario acessa app.seudominio.com, hospedado no Amplify.
- Frontend chama api.seudominio.com, hospedado no App Runner.
- API valida JWT proprio, aplica regras de empresa/funcionario e acessa o RDS PostgreSQL.
- Logs do backend ficam no CloudWatch e variaveis sensiveis ficam fora do codigo.

Passo 1 - Ajustar o frontend

Altere o arquivo frontend/src/services/api.js para usar uma URL de API configuravel por ambiente:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3000"
})
```

No Amplify, defina VITE_API_URL com a URL do App Runner ou do dominio api.seudominio.com.

Passo 2 - Preparar o repositorio

1. Suba o projeto para o GitHub em uma branch principal, como main.

2. Confirme que `.env` esta no `.gitignore` e nao foi enviado ao repositorio.
3. Inclua migrations do Prisma no repositorio, pois elas serao aplicadas em producao.

Passo 3 - Criar o RDS PostgreSQL

1. Acesse RDS > Create database.
2. Escolha PostgreSQL e defina o nome do banco, por exemplo `sistemaasaas`.
3. Use senha forte e armazene a credencial em local seguro.
4. Ative backups automaticos.
5. Prefira Public access = No para producao.

A `DATABASE_URL` final tera o formato:

```
postgresql://usuario:senha@endpoint-rds:5432/sistemaasaas?schema=public
```

Passo 4 - Configurar seguranca do banco

- O RDS deve aceitar conexoes na porta 5432 apenas do backend/App Runner.
- Evite liberar 0.0.0.0/0 para o banco.
- Se precisar aplicar migrations da sua maquina, libere temporariamente apenas o seu IP e remova depois.

Passo 5 - Aplicar migrations

Com a `DATABASE_URL` de producao configurada no ambiente local ou no pipeline, rode:

```
npx prisma migrate deploy
npx prisma generate
```

Em producao, use `migrate deploy`. Evite `prisma migrate dev` no banco real.

Passo 6 - Criar variaveis e segredos

Configure no App Runner ou em SSM/Secrets Manager:

- `DATABASE_URL`: string de conexao do RDS.
- `JWT_SECRET`: valor longo, aleatorio e forte.
- `FRONTEND_URL`: URL do Amplify ou `app.seudominio.com`.
- `NODE_ENV=production`.
- Credenciais de email, se o reset de senha estiver ativo.

Passo 7 - Subir backend no App Runner

1. Acesse App Runner > Create service.
2. Escolha Source code repository e conecte ao GitHub.
3. Selecione o repositorio e branch main.

4. Runtime: Node.js na versao suportada mais recente.
5. Build command: `npm ci && npx prisma generate`.
6. Start command: `npm start`.
7. Porta: 3000.
8. Configure as variaveis de ambiente e, se necessario, VPC Connector para acessar o RDS privado.

Ao final, o App Runner entregara uma URL HTTPS. Ela sera usada temporariamente como `VITE_API_URL`.

Passo 8 - Testar o backend

- Abra a URL do App Runner e teste `/teste`. Como exige token, uma resposta de token ausente ja confirma que a API esta respondendo.
- Confira App Runner > Logs para erros de `DATABASE_URL`, Prisma ou variaveis ausentes.
- Teste cadastro, login e uma rota autenticada com o frontend local apontando para a API de producao.

Passo 9 - Subir frontend no Amplify

1. Acesse Amplify Hosting > Deploy an app.
2. Conecte ao GitHub e selecione o repositorio.
3. Defina a subpasta frontend como app root.
4. Build command: `npm ci && npm run build`.
5. Output directory: `dist`.
6. Adicione `VITE_API_URL` com a URL do App Runner ou `api.seudominio.com`.

Passo 10 - Configurar rotas SPA

Como o app usa React Router, configure rewrite para paginas internas nao darem 404 ao atualizar:

```
source: </^[^.]>+$/>
target: /index.html
status: 200
```

Passo 11 - Configurar CORS

No backend, defina `FRONTEND_URL` exatamente com a URL publica do frontend:

- Antes do dominio: `https://seu-app.amplifyapp.com`.
- Depois do dominio: `https://app.seudominio.com`.

Passo 12 - Dominio proprio

Sugestao de organizacao:

Area	Servico recomendado
app.seudominio.com	Aponta para o Amplify Hosting.
api.seudominio.com	Aponta para o App Runner.

Depois de ativar os dominios, atualize VITE_API_URL para <https://api.seudominio.com> e FRONTEND_URL para <https://app.seudominio.com>.

Checklist antes de abrir para clientes

OK	Item	O que verificar
	JWT_SECRET forte	Valor longo, secreto e fora do código.
	HTTPS ativo	Frontend e backend acessíveis somente por HTTPS.
	RDS com backup	Backups automáticos e snapshots configurados.
	CORS restrito	Somente o domínio real do frontend.
	Migrations aplicadas	prisma migrate status sem pendências.
	Fluxo testado	Cadastro, login, clientes, serviços, produtos, vendas, agendamentos e comissões.
	Logs revisados	CloudWatch sem erros recorrentes.
	Email validado	Recuperação de senha funcionando, se aplicável.

Proximas etapas do SaaS

- Criar modelo de planos e status de assinatura na tabela Empresa.
- Adicionar trial, data de vencimento e bloqueio por inadimplência.
- Integrar pagamento com Stripe, Mercado Pago ou Pagar.me.
 - Criar painel administrativo interno para acompanhar empresas assinantes.

Ordem recomendada de execucao

1. Ajustar VITE_API_URL no frontend.
2. Subir código no GitHub.
3. Criar RDS PostgreSQL.
4. Aplicar migrations no banco de produção.
5. Criar secrets e variáveis.
6. Subir backend no App Runner.
7. Subir frontend no Amplify.
8. Configurar CORS e domínios.
9. Executar testes completos como admin e funcionário.